

INDEX

NAME Manosi Saha SUBJECT Mobile Security

STD. _____ DIV. _____ ROLL NO. _____ SCHOOL NFSU, Chennai

Sr. No.	Page No.	Title	Date	Teacher's Sign / Remarks
01	21/01/26	Installation of Android Studio and creating a Hello world Application.		R.Vij
02	28/01/26	Security testing of tiva app using ADB.		R.Vij
03	04/02/26	Static Analysis of Mobile Application using MobSF.		R.Vij
04	11/2/26	Dynabnc Analysis of Mobile Application using MobSF.		R.Vij
05	18/2/26	Reverse Engineering of android application using APK Tool, Hex Dump & DexDump.		R.Vij
06	18/03/26	Security Auditing of Android application using Drozer.		R.Vij
07	26/03/26	Performing Android application vulnerability assessment using OARK.		R.Vij
08	01/4/26	Analysis of network Traffic for android device using HTTP toolkit.		R.Vij

PRACTICAL - 01

01. AIM: To install Android Studio on windows System & create & run basic Hello world Android Application

02. PROCEDURE:

Step 1: System Requirements

a. Ensure the system meets the following:

- Window 10/11 64 bit
- Minimum 8 GB RAM
- Atleast 10-15 GB free disk space
- Virtualization enabled in BIOS.

Step 2: Download Android Studio

- Open web browser
- Visit the official website
- click Download Android Studio
- Accept the terms & Download the installation.

Step 3: Install Android Studio

- Run the download .exe file
- click Next
- Select components:-
 - Android studio
 - Android Virtual Devices (AVD)
- choose installation location.
- click Install and wait for completion
- click Finish to launch Android studio.

Step 4: Initial Setup

- Android Studio Setup wizard will open
- Select Standard Installation.
- choose a UI Theme (Light / Dark)
- choose Allow to download:
 - Android SDK
 - Emulator components.
- click Finish after Setup completes.

Step 5: Create Hello world Application

- a. Open Android Studio
- b. click New Project.
- c. Select Empty Activity and click Next.
- d. Enter the following Details:
 - Name: Hello world
 - Package Name: com.example.helloworld
 - Language: Java or Kotlin
 - Minimum SDK: API 21 or higher
- e. click Finish
- f. Wait for Gradle build to complete

Step 6: Run the Application

- a. Enable Develop option on phone.
- b. Enable USB Debugging.
- c. connect device ~~via~~ USB.

Step 7: Execute the App.

- a. click the Run button.
- b. Select Connected device.
- c. Wait for installation & launch.

03. RESULT:

- a. Android studio was Successfully installed on windows.
- b. A Hello world application was created, compiled & executed Successfully on a physical device, displaying the message "Hello world" on the screen.

R. J. 7

PRACTICAL - 02

01. AIM: TO analyze the DNA app using ADB and identify security vulnerabilities such as insecure logging, hardcoded information etc.

02. PROCEDURE:

Step 1: Setup Environment

a. Install Android Emulator

b. Enable Developer options → USB Debugging

c. Verify adb connection:

`adb devices`

d. Install DIVA APK:

`adb install diva-beta.apk`

Step 2: Launch the application and start Vulnerability testing.

(a) Insecure logging:

- open log monitoring

`adb logcat`

- Perform actions inside DIVA (login, data entry)

- Filter logs:

`adb logcat | findstr diva`

- Observe for:

- usernames / Passwords in logs

- Sensitive data printed.

- Debug messages exposing internal logic.

~~Step~~ (B) Hardcoded Information Detection

- Pull the APK:

`adb shell pm path jakhar.aseen.diva`

`adb pull <apk -path>`

(b) Extract readable strings such as password, key, token
Strings base.apk | grep -i password
Strings base.apk | grep -i key
Strings base.apk | grep -i token.

(d) Search for hardcoded values:

- API keys
- Credentials
- Secret strings

(c) Input Validation:

- Open Input validation challenge
- Enter unexpected inputs such as:
 - SQL injection payload '1 OR 1=1 --
- Observe application behaviour.

(d) Access Control:

- List app activities
adb shell dumpsys package jakhar.assem.diva |
findstr Activity.
- Attempt to launch restricted activities directly
adb shell am start -n jakhar.assem.diva /.
<ActivityName>
- As observed restricted screen opens without authentication.

03.

RESULT: ✓

- Sensitive information was observed in logcat, indicating insecure logging.
- Hardcoded credentials & secrets were identified within the application.

R.V.

PRACTICAL - 03

01. AIM : To perform Static Analysis of an Android app using Mobile Security Framework (MobSF).

02. PROCEDURE :

Step 1: Install Mobile Security Framework from github

```
git clone https://github.com/MobSF/Mobile-Security-Framework  
-rk -MobSF-git
```

Step 2: create a virtual Environment inside MobSF directory.

```
python -m venv mobsf
```

Step 3: Install all the required dependencies using below command.

```
For Linux :- ./setup.sh
```

```
For windows:- Setup.bat
```

Step 4: Run the MobSF using below command

```
For Linux :- ./run.sh 127.0.0.1:8080
```

```
For Windows :- run.bat 127.0.0.1:8080
```

Step 5: Open browser and search 127.0.0.1:8080

once login page comes login with default credentials

username : mobsf

Password : mobsf

Step 6: For uploading the application click on "upload & Analyze" or Drag & Drop the file.

Step 7: After uploading the application automatically detects the app type and analyse the application.

Step 8: Analyze Application Permissions

- Dangerous permissions (camera, SMS)
- Over-privilege permissions.

Step 9: Analyze android APIs

Step 10: Manifest Analysis.

- view Android Manifest.xml
- Identify
 - Exploited activities.
 - Debuggable flag.
 - Backup enabled.

Step 11: Code Analysis

- Review Java / smali Code
- Identify
 - Hardcoded credentials
 - API keys
 - Insecure Functions.

Step 12: Security Issues Detection

MobSF highlights:

- weak cryptography
- Insecure Storage
- Improper SSL configuration
- logging Sensitive Data.

Step 13: View Report.

- Generate PDF / HTML Report.

- Review risk levels:

- (a) High

- (b) Medium

- (c) Low

03. RESULT:

(a) Have Successfully performed Static analysis using MobSF.

(b) Multiple vulnerabilities insecure permissions, hardcoded credentials & weak configurations were identified without executing the application.

R. U. A

PRACTICAL - 04

01. AIM: To perform Dynamic Analysis of Mobile app using Mobile Security Framework (MobSF)

02. PROCEDURE:

Step 1: Install Genymotion & create a virtual device (Android 8/9 recommended)

Step 2: Wait until emulator fully boots.

Step 3: Start the MobSF using below command
`run.bat 127.0.0.1:8080`

Step 4: Connect to the virtual device with below commands.

`adb devices`

`adb connect <IP Address>`

Step 5: Uploaded Damn Vulnerable Bank app Apk in MobSF & wait for static analysis to complete.

Step 6: Once Static Analysis is completed click on "Dynamic Analyzer" tab.

Step 7: MobSF will:

- Detect Genymotion emulator.
- Install MobSF agent.
- Prepare runtime monitoring.

Step 8: App will install automatically then open the app and perform actions:

- login
- check Balance
- Transfer money.

Step 9: Observe in MobSF:

- API calls
- Network Requests
- File System access
- Logcat logs.

Step 10: Once the analysis is completed click on generate report.

Step 11: Review vulnerabilities:

- High, medium, low.

03. RESULT :

The dynamic analysis of Damn Vulnerable Bank application using MobSF with Genymotion emulator Successfully identified multiple runtime vulnerabilities.

(a) Sensitive Data exposed in API responses.

(b) Improper SSL certificate validation.

(c) Weak authentication mechanisms.

(d) Sensitive information logged in logcat.

(e) Potential data leakage in local storage.

R.V

PRACTICAL - 05

01. AIM: To Reverse Engineer an android application using APK Tool, Hex Dump and Dek Dump.

02. PROCEDURE:-

Step 1: Install required tools using below command

```
Sudo apt install apktool dexdump
```

Step 2: Verify Installation:

```
apktool  
dexdump
```

Step 3: Download the APK file to analyze and for reverse engineering & place it in working directory.

Step 4: Decompile the Apk file using APK Tool

```
apktool d <Apk-File>
```

Step 5: Navigate to new folder created.

```
cd diva - beta  
ls
```

Step 6: You will observe:

(a) AndroidManifest.xml

(b) Smali /

(c) res /

(d) assets /

Step 7: Analyze AndroidManifest.xml file

```
cat AndroidManifest.xml
```

Observe:-

- (a) Exported activities (vulnerable entry point)
- (b) Permissions like:
android.permission.Internet
- (c) Debuggable flag may be enabled.

Step 8: Analyze Smali code by navigating to below folder

cd smali/jakhar/aseem/diva

Step 9: open a file:

nano HardcodeActivity.smali

Look for:

- (a) Hardcoded credential.
- (b) Insecure logic.
- (c) Logging statements.

Step 10: Perform Hex Dump Analysis using Below command

Hexdump -C diva-beta.apk > output.txt

Observe:

- (a) APK starts with PK (ZIP signature)
- (b) Embedded readable strings.
- (c) Metadata & binary structure.

Step 11: Extract classes.dex for analyzing it using dex dump

unzip diva_beta.apk

Step 12: Analyze using Dex Dump using below command
dexdump -d classes.dex > dex.txt.

Search for:

/class.

/method.

Step 13: Extract Strings using below command.

Strings classes.dex > String.txt

Step 14: Search for Sensitive data keywords like

(a) API

(b) Password

(c) Secret.

(143) Result:

(a) The DIVA APK was Successfully reverse engineering using APK Tool, Hex Dump & Dex Dump tools.

(b) The analysis revealed insecure coding practices & vulnerabilities like.

(.) Hardcoded credentials.

(.) Application logic.

(.) Insecure coding practices.

Ravi

PRACTICAL - 06

01. AIM: To perform security auditing of an android application using Drozer.

02. PROCEDURE :

01. Install Drozer using below command
`pip install drozer.`

02. Start Genymotion Emulator & check for the emulator
`adb devices.`

03. Install diva apk into emulator
`adb install diva-beta.apk`

04. Verify the installation using below command
`adb shell pm list packages | findstr diva`

05. Download Drozer agent apk from github repository & install the apk in emulator
`adb install drozer-agent.apk`

06. Open the app inside emulator & enable
"Embedded Server"

07. Setup port forwarding using below command
`adb forward tcp:31415 tcp:31415`

08. connect to drozer

drozer console connect.

09. Identify installed packages

run app.package.list -f diva

10. Get package information

run app.package.info -a jakhar.aseem.diva

check: permissions

UID

Debuggable flag.

11. Identify attack surface:

run app.package.attack surface jakhar.aseem.diva

This displays:

(a) Activities

(d) Broadcast Receives.

(b) Services

(e) Content Providers.

12. Enumerate Activities:

run app.activity.info -a jakhar.aseem.diva

And look for:

(a) exported = true.

(b) No permission protection.

13. Launch Vulnerable Activity

run app.activity.start --component jakhar.aseem.diva
jakhar.aseem.diva.API credsActivity.

14. Analyze content providers

```
run app.provider.info -a jakhar.aseem.diva
```

15. To query data

```
run app.provider.query Content://jakhar.aseem.  
diva.provider.notesprovider/notes.
```

16. Analyze broadcast receivers

```
run app.broadcast.info -a jakhar.aseem.diva
```

03. RESULT:

We have successfully performed security auditing of DIVA apk using Drozer.

The analysis identified vulnerabilities such as:

(a) Exported components.

(b) Insecure storage.

(c) Unprotected content provider.

PRACTICAL - 07

01. AIM: To analyze Android Application (APK) identify potential security vulnerabilities using the QARK tool.

02. PROCEDURE:

Step 1: Setup Environment by cloning QARK using below command.

```
git clone http://github.com/xtiankisutsa/MARA-framework.git
```

Step 2: Create a python 3 virtual environment
`python -m venv quarkenv`

Step 3: Activate the virtual environment
`Source quarkenv/bin/activate`

Step 4: Navigate to quark directory.
`cd MARA-Framework/tools/quark`

Step 5: Install all dependencies
`pip install -r requirement.txt`

Step 6: Set python path for quark using below commands.

```
echo $SHELL
```

```
if [ $SHELL = zsh ]
```

```
then nano ~/.zshrc
```

. At end of file add below line

```
export PYTHONPATH = $PYTHONPATH:/home/nano/  
MARA-Framework/tools/qark
```

- Save the file & exit, after that run below command.

```
Source ~/.zshrc.
```

- Run qark using below command

```
python2 qark/qarkMain.py
```

7. Run qark using below command

```
python2 qark/qarkMain.py
```

8. Android SDK setup:- when prompted for downloading Android SDK hit "Y" & enter.

9. Select "1" for examining APK file.

10. Provide full path of the APK file.

```
/home/nano/Downloads/diva-beta.apk
```

11. QARK will Perform

- (a) Static code analysis
- (b) Manifest analysis
- (c) vulnerability detection
- (d) Security misconfiguration checker.

12. After completion QARK generator

- (a) vulnerability report.
- (b) Security issues list.
- (c) Risk classification.

13. The results includes:

- (a) Insecure data storage
- (b) Hardcoded credential
- (c) Exported components.
- (d) weak cryptographic usage.

RESULT:

- (a) we have Successfully analyzed a android application using QARK
- (b) Multiple security vulnerability were identified including insecure data storage, exported components & improper cryptographic Implementation.
- (c) QARK generated a detailed report highlighted potential risks & remediation suggestions.

RUF

PRACTICAL - 08

01. AIM: To perform Network Traffic Analysis for android device using HTTP Toolkit.

PROCEDURE:

02. Step 1: Download & install HTTP toolkit from its official website.

Step 2: Launch HTTP toolkit & Select "Android Device via ADB" from the intercept option.

Step 3: Make sure android device & System are on same network. confirm it using below command

adb devices

Step 4: HTTP toolkit will automatically configure the proxy setting on the connected device.

Step 5: HTTP toolkit will detect the device & ~~object~~ inject its CA certificate to intercept HTTPS traffic.

Step 6: In android device open any browser.

Step 7: Navigate to <http://testaspnet.vulnweb.com/login.asp>

Step 8: The login page of the vulnerable web application will load.

Step 9: Enter a test username (e.g. admin) in the username field.

Step 10: Enter a test password (e.g. admin) in the password field.

Step 11: Click the login/submit button.

Step 12: Switch to HTTP toolkit & observe a new HTTP POST request to /login.aspx to open its details.

Step 13: In the body section observe the form data containing:

tbusername : admin

tbPassword : admin

Step 14: The credentials are transmitted in plain text (no encryption) confirming the vulnerability.

03. RESULT:

(a) we have successfully intercepted HTTP/HTTPS traffic from an android device using HTTP toolkit.

(b) observed & analyzed APT requests, POST requests headers & responses.

(c) Identified potential security issues such as:

- unencrypted data transmission.
- The request body clearly contained the username & password in plain-text.